Chasing a personal record is a complex goal for many marathon runners, involving months of planning and training before culminating in an hours-long race against their past selves. There are many variables at play when trying to attain a new PR, including race conditions, the runner's training schedule, ability to avoid injuries, and adherence to proper hydration and nutrition. In this project, I aim to identify which of the many factors at play best predict a runner's chance to beat their PR. Due to limitations in my data set, I narrowed my focus to experienced marathon runners who have successfully completed at least one marathon and use machine learning models to explore the factors that best predict a repeat marathon runner's ability to achieve a new PR.

The data set I selected is the Run Club Marathon Performance Dataset from Kaggle, a synthetic data set designed to simulate real-world marathon data. This data set has 100,000 observations and includes 40 variables. Originally, I intended to examine the relationship between a runner's target finish time and actual finish time to investigate the factors that enabled runners to achieve their time goal. However, I discovered that there was not a single instance in the full 100,000 observations where a runner accomplished their goal. This is a limitation of my dataset, as this is not reflective of real world data. The number of runners who beat their time goal may be few, but 0 among a large dataset seems unrealistic. From there, I pivoted my project to focus on a runner's ability to achieve a new PR. To do this, I defined a new variable:

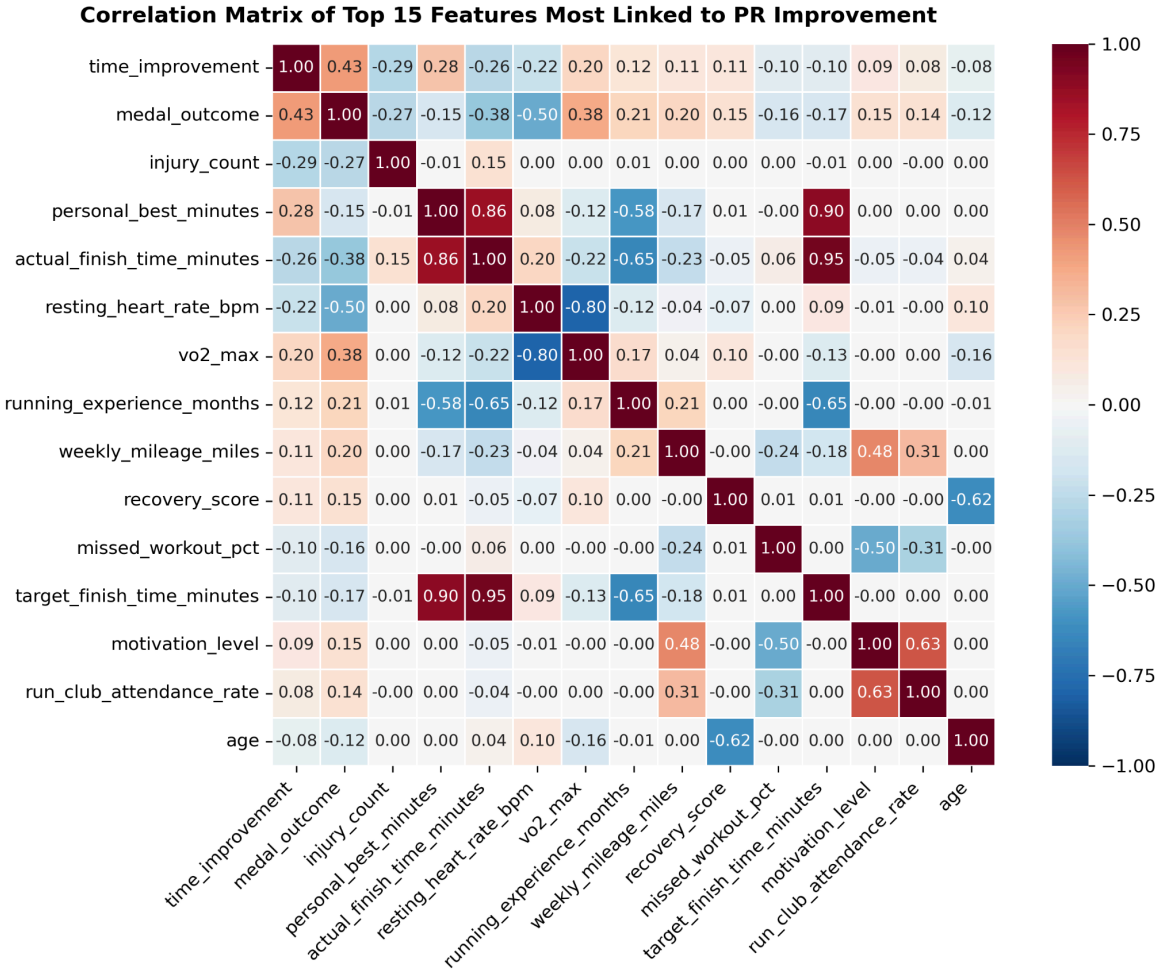
Time improvement = personal best minutes - actual finish time minutes.

However, this approach presented a new challenge: all first time marathon runners do not yet have a PR to construct the outcome variable. While some machine learning models can adapt to missing input variables, missing outcome variables are much more problematic. This led me to restrict my question to focus on repeat marathon runners and drop the 13,416 observations of first time marathon runners. Still, there was another hurdle with my approach, as there were 2,149 observations where there was no data for the actual finish time in minutes. These observations also had to be dropped so that there were no missing data in the outcome variable, time improvement. Unfortunately, these observations contained interesting information where runners may have suffered from injury, been ill-hydrated, faced adverse weather conditions, or did not finish for a variety of other reasons that my models will not be able to capture. After dropping these two groups of runners, there were 84,435 observations in my dataset. The missing observations of actual finish time before they were dropped, along with other variables containing missing data points, are displayed in the table below:

Feature Name	Data Type	Missing Values (N)	Missing %	Variance (Std Dev)
injury_severity	object	51292	59.2%	N/A
medal_outcome	float64	17329	20.0%	0.80
vo2_max	float64	10431	12.0%	8.50
cross_training_hours_per_week	float64	8634	10.0%	2.37
nutrition_score	float64	6921	8.0%	2.21
hydration_consistency	float64	6851	7.9%	2.11
sleep_hours_avg	float64	4350	5.0%	1.00
actual_finish_time_minutes	float64	2149	2.5%	33.62
time_improvement	float64	2149	2.5%	18.10
weekly_mileage_miles	float64	0	0.0%	7.06
training_streak_days	int64	0	0.0%	21.60
gender	object	0	0.0%	N/A
mental_preparation_score	int64	0	0.0%	2.07
running_experience_months	int64	0	0.0%	35.36
target_finish_time_minutes	int64	0	0.0%	30.37

After examining the data, I believe that the missing values in injury severity actually represent individuals who did not suffer an injury and replaced the nan values with “No injury” flags. Other missing values are addressed in my data and cleaning preparation steps. I dropped medal_outcome as I aimed to predict time improvement across slow and fast runners alike. I also dropped the marathon date and runner ID from the dataset, as I felt they were unrelated to time improvement and might lead to spurious results. Finally, I also dropped weekly mileage in kilometers as I already have the same data captured in miles.

I prepared the data the same for all three models, utilizing sklearn’s standard scaler for numerical data points and sklearn’s one-hot encoder for categorical variables. This standard scaler is necessary for the MLP Regressor and Elastic Net models to properly function. The one-hot encoder was also necessary for MLP Regressor and Elastic Net. XGBoost does not require a standard scaler and can natively handle categorical data, but I kept the preprocessing the same for all three for simplicity and ease of comparison. As part of my data preparation pipeline, I utilized an imputer using the median value for numerical values and the most frequent values for the categorical variables. Also, in my data preparation steps, I dropped time improvement and actual race times from my feature data sets. I elected to drop actual race times as this, in combination with personal best, would be a linear combination of the outcome variable. Finally, I utilized sklearn’s test train split functionality to split 20% of the data into a test data set. For this split, I used a random state of 7905 for reproducible results.



To address my research question, I implemented two main machine learning models as well as a third simpler model to offer a point of comparison. The first main model was Xtreme Gradient Boosting (XGBoost hereafter), and the second main model was Multi-Layer Perceptron Regressor (MLP Regressor hereafter). The third, simpler, model is Elastic Net. All three of these models are supervised learning algorithms.

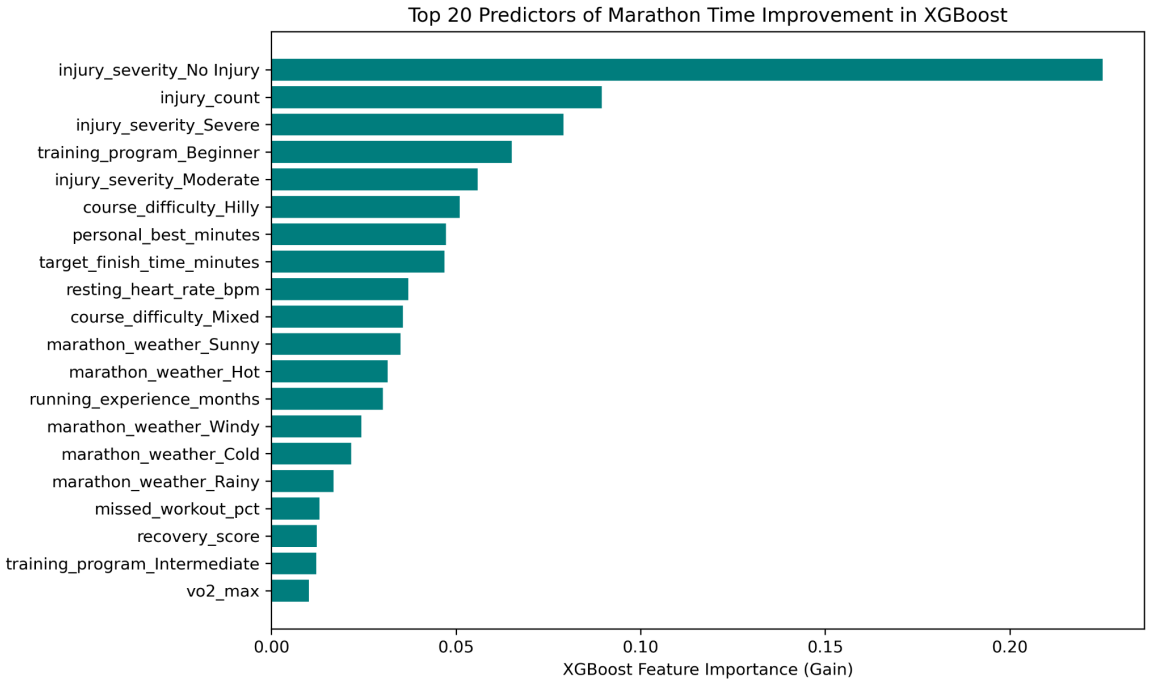
XGBoost is a gradient descent model that utilizes an ensemble learning algorithm to progressively "boost" weak learners as it seeks to minimize the loss function (CITE). My model implementation uses decision trees as weak learners with a loss function that minimizes the mean squared error of time improvement. I chose XGBoost due to its raw predictive power and ability to handle my large data set. Additionally, given that I have over 35 predictive features even before the one-hot encoder, I preferred XGBoost over the simpler gradient boosting algorithm due to its regularization abilities. XGBoost also has the advantage of natively handling categorical data and adapting to missing data in the features. However, I did not utilize these advantages as I used the same data preprocessing steps for all three of my models for simplicity and comparability.

The initial parameters I used for the XGBoost model were 500 trees, a max tree depth of 6, a learning rate of 0.05, a subsample of 0.8, and a colsample by tree of 0.8. I initially had R2 values over 99% and was concerned about overfitting given the power of the XGBoost model. I experimented with all of the different parameters listed, but settled on all the same parameters except for decreasing the learning rate to 0.0025. A more advanced project would do a more methodical exploration of these parameters.

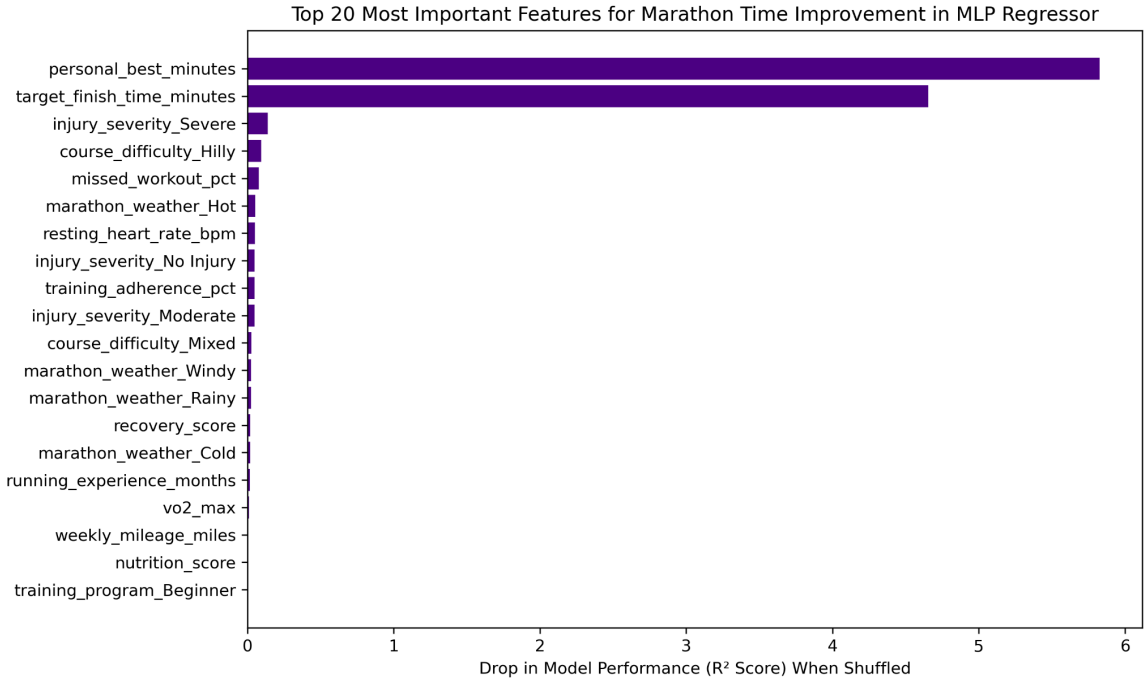
MLP Regressor is a simple neural network model that uses backpropagation to learn and update the weights of the hidden layers. I chose MLP Regressor to differentiate from the decision tree-based model algorithm of XGBoost, which also offers a high degree of predictive power for a regression model. Once again, I had concerns about overfitting for this model. I attempted to address this with an early stopping function where the model finishes early if improvements stop being made of 10 epochs. However, I could further address this concern by using only one hidden layer and implementing drop out. As an improvement to this project, I would experiment with the model's hidden layers. The parameters I used for MLP Regressor were two hidden layers, the first with 64 nodes and the second with 32 nodes. I used a maximum number of epochs of 200. I used the default activation function, rectified linear unit (ReLU), for its computational speed, given my large dataset. For the optimization function, I used Adaptive Moment Estimation (Adam) for its speed and adaptability of the learning rate.

Finally, I implemented a simple Elastic Net regression algorithm to provide a contrast to the other two models and highlight the relative strengths and weaknesses. Elastic Net has the advantages of both Lasso and Ridge regularization, to help address the problem of both unimportant features and correlated features at the same time, which is key given my large number of features. The parameters I used for Elastic Net were an alpha of 1 and a ridge penalty of 0.5. An improvement on this model would be to use grid search to find the optimal parameters. As I will show below, Elastic Net yields significantly weaker results compared to the two main models and serves primarily to juxtapose the high power of the XGBoost and MLP Regressor models.

The most important predictors of marathon time improvement according to the XGBoost model were avoiding injury, with the top three predictors all being related to injuries. As shown in Figures 1 and 3 in the appendix, having no injury was a positive predictor for time improvement, while having a non-zero injury count or a severe injury were negative predictors. After that, running experience and race conditions were among the strongest predictors of time improvement in the XGBoost results.



The MLP Regressor model shared some of the results from the XGBoost model, but had personal best and target finish time as the most dominant predictors of the model. While injury and course difficulty variables were among the most important features, measured by the drop in R2 values after shuffling the values, the runner's present record and targeted finish time were much stronger predictors. In Figure 2 in the appendix, the steepness of the partial dependence plots for personal best and targeted finish time corroborates the findings in the chart below. Injury severity and course difficulty are significantly flatter compared to the top two features.



The MLP Regressor model performed the best across mean average error, root mean squared error, and R2. The R2 value indicates that the model captures over 99% of the variance in the data. However, XGBoost was not far behind with MAE and RMSE values roughly double that of the MLP Regressor model and a very strong R2. In stark contrast is the Elastic Net model, with MAE and RMSE values over ten times that of the MLP model. The Elastic Net model also scored significantly lower for R2, with the model capturing 37% of the variation in the data. This highlights the raw predictive power of the XGBoost and MLP Regressor models.

Model Name	MAE (Minutes)	RMSE (Minutes)	R^2 Score
XGBoost Regressor	1.82	2.47	98.12%
MLP Regressor	0.80	1.36	99.43%
Elastic Net	11.41	14.31	36.91%

Interestingly, despite the large differences in model metrics, the MLP Regressor and Elastic Net models selected similar variables, aligning on two of the top three features and five of the top 10 most important features. XGBoost scored very similarly to MLP Regressor in MAE, RMSE, and R2, and agreed on 7 of the top 10 features. Overall, a higher personal best, lower target finish time, lack of a severe injury, lower injury count, flat course, and lower resting heart all were among the top 10 most important predictive features across all three models. This alignment supports the model results.

Rank	XGBoost Regressor	Neural Network (MLP)	Elastic Net Baseline
1	injury_severity_No Injury	personal_best_minutes	personal_best_minutes
2	injury_count	target_finish_time_minutes	target_finish_time_minutes
3	injury_severity_Severe	injury_severity_Severe	injury_count
4	training_program_Beginner	course_difficulty_Hilly	resting_heart_rate_bpm
5	injury_severity_Moderate	missed_workout_pct	running_experience_months
6	course_difficulty_Hilly	marathon_weather_Hot	injury_severity_No Injury
7	personal_best_minutes	resting_heart_rate_bpm	course_difficulty_Hilly
8	target_finish_time_minutes	injury_severity_No Injury	vo2_max
9	resting_heart_rate_bpm	training_adherence_pct	recovery_score
10	course_difficulty_Mixed	injury_severity_Moderate	weekly_mileage_miles

In conclusion, the two most important factors in predicting if an experienced marathon runner will achieve a new PR according to the above models are their starting point and their ability to avoid injury. The first is fairly obvious; if a person aims to decrease their marathon time by 10 minutes, it will be much easier to do so if their PR is 5 hours than if it is 3 hours. Similarly, the targeted finish time is also a strong predictor, but likely for the same reason, as people's goals will be incrementally better than their past performance. More interesting are the results on the role of injury as the runner has more near-term control in avoiding injury. Having no injury was a strong positive predictor, while moderate and severe injuries, as well as the number of injuries, were all negative predictors of a time improvement. Weekly mileage and training adherence were among the stronger indicators of time improvement, but lacked the overwhelming consensus between the three models that injury avoidance did. It may be better to prioritize targeted strength training and recovery as opposed to adding extra miles to a training block if the goal is to improve the race time. There is plenty of research indicating that a gradual increase in mileage, targeted strength training, and prioritizing recovery and sleep, among other strategies, can reduce the risk of injury (Gao & Finnoff 2021, Lauersen et al. 2014, Nielsen et al. 2014).

My tech stack for this project consisted of Jupyter Notebook and Python files in VS Code, my local CLI, and Docker, as well as Gemini for troubleshooting and coding help. I used Jupyter Notebook in

the early stages for data exploration and testing out the models for quick and intermediate feedback. Once my project became more developed, I began running a Python script in my CLI. Finally, I moved on to running Docker locally. I originally intended to use GCP to run my final code version, but I found that my code ran fairly quickly, and moving to GCP would not be a big advantage. Additionally, I used Gemini as my preferred LLM to help me find my dataset, write code, and troubleshoot issues I ran into along the way. Gemini was extremely helpful in completing this project on a shortened timeframe. Though Docker was not immediately helpful to me, if I were to expand this project in the future to a more extensive group project, Docker would be greatly helpful to coordinate across one working code base.

Citations & AI Use Declaration:

IBM. (2025, April 8). *What is XGBoost?* IBM Think Topics. https://www.ibm.com/think/topics/xgboost

IBM Technology. (2022, July 11). *What are MLPs (Multilayer Perceptrons)?* [Video]. YouTube. https://www.youtube.com/watch?v=7YaqzpitBXw

Econoscent. (2020, October 10). *Visual Guide to Gradient Boosted Trees (xgboost)* [Video]. YouTube. https://www.youtube.com/watch?v=TyvYZ26alZs

Gao, B., & Finnoff, J. T. (2021). Sleep and injury risk. Current Sports Medicine Reports, 20(6), 303–306. https://doi.org/10.1249/JSR.0000000000000849

Lauersen, J. B., Bertelsen, D. M., & Andersen, L. B. (2014). The effectiveness of exercise interventions to prevent sports injuries: A systematic review and meta-analysis of randomised controlled trials. British Journal of Sports Medicine, 48(11), 871–877. https://doi.org/10.1136/bjsports-2013-092538

Nielsen, R. O., Parner, E. T., Nohr, E. A., Sørensen, H., Lind, M., & Rasmussen, S. (2014). Excessive progression in weekly running distance and risk of running-related injuries: An association which varies according to type of injury. Journal of Orthopaedic & Sports Physical Therapy, 44(10), 739–747. https://doi.org/10.2519/jospt.2014.5164

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Ritvikmath. (2021, September 29). *Gradient Boosting : Data Science's Silver Bullet* [Video]. YouTube. https://www.youtube.com/watch?v=en2bmeB4QUo

AI declaration: Throughout this project I used Gemini to help me find the data set, brainstorm, troubleshoot, and write code. The discussion is written in my own words.

Appendix

Figure 1.

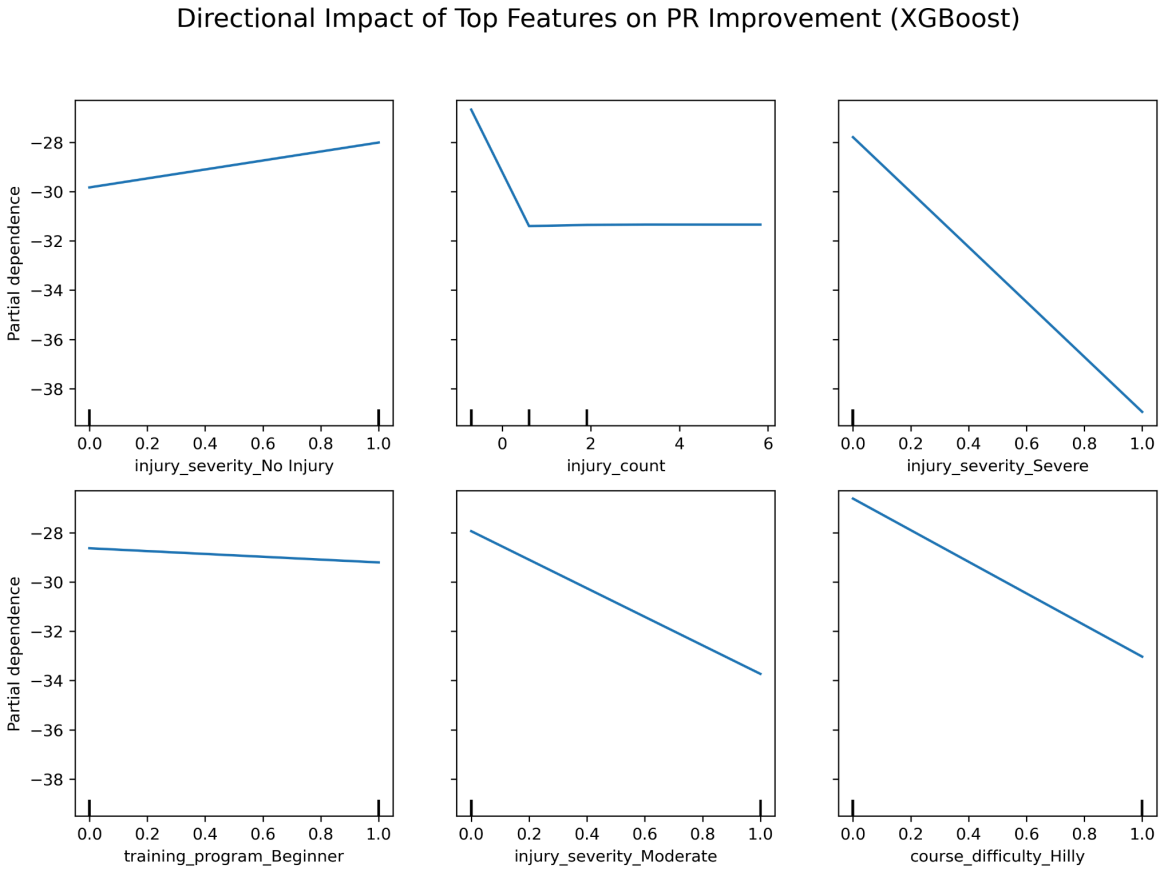


Figure 2.

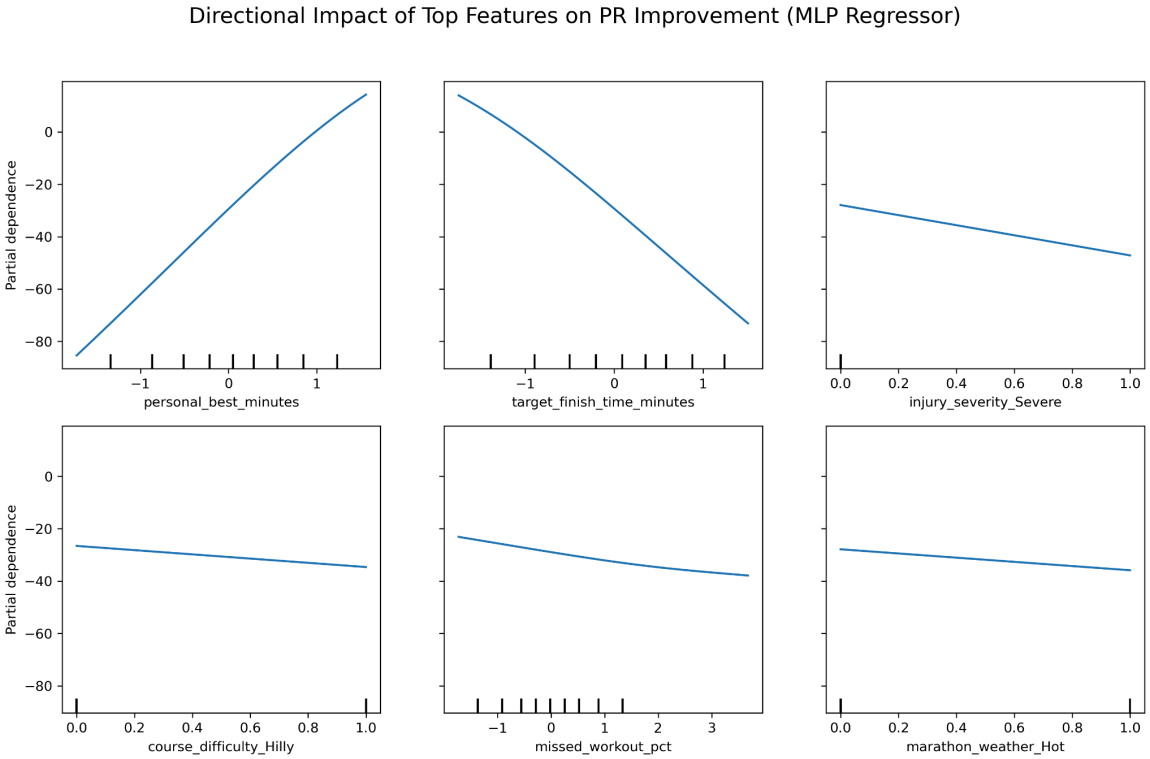


Figure 3.

Feature Name	Pearson Correlation (r)
injury_count	-0.29
personal_best_minutes	0.28
resting_heart_rate_bpm	-0.22
vo2_max	0.20
running_experience_months	0.12
recovery_score	0.11
missed_workout_pct	-0.10
target_finish_time_minutes	-0.10